

クラウド LLM とローカル LLM を プライバシー境界で切り分ける LLM 呼び出しフレームワーク詳細比較レポート

プロンプトインジェクション／GitLost 脅威モデル統合改訂版

2026 年 7 月 20 日

統合改訂の要点

元レポートの比較軸・章立てを維持しながら、間接プロンプトインジェクションを独立した脅威モデルとして追加した。評価項目には、Prompt Injection 検出、Instruction/Data 分離、Tool Capability 制御、Provenance/Taint 伝播、Sink Clearance、最終 Egress 再検査、累積漏洩制御を追加し、推奨アーキテクチャを「侵害された LLM を前提にしても漏洩しない」構造へ拡張している。

統合版

エグゼクティブサマリ

本調査で挙げた候補は、実態として同じ種類の「LLM オーケストレーション基盤」ではない。大きく分けると、データ境界と参照整合性を扱うレジストリ型の NBAccess / SourceVault、実運用のゲートウェイやエージェントのフックで再検査するルータ型の ClawRouter / EdgeClaw と vLLM Semantic Router、プライバシー性能トレードオフを研究する推論方式の PRISM / Privacy-R1 / PrivacyRestore、ローカル匿名化やローカル RAG で外部送信を減らす AgentStealth / SEAL / EvidenceBot / SecureLLM、評価・匿名化ツール群の ConfAlde / PrivacyLens / IBM AI Privacy Toolkit に分かれる。これらは排他的な代替品ではなく、組み合わせるべきレイヤである。

ファイル移動・コピー・別経路参照でラベルが脱落する漏洩に対しては、NBAccess / SourceVault と ClawRouter / EdgeClaw が比較的強い。前者は ObjectSpec、PrivacyLevel、content hash、immutable snapshot、append-only event log、bundle 依存関係によって「場所」より「データと由来」に寄せられる。後者はメッセージ、tool call、tool result、永続化、送信直前を複数の Hook で再検査し、dual-track memory / session によってクラウド側履歴を脱機微化するため、上流の分類ミスを下流で吸収しやすい。

本統合版では、これに加えてプロンプトインジェクション、とくに GitLost 型の間接プロンプトインジェクションを主要な脅威モデルとして扱う。重要なのは、Prompt Injection Detector の精度を上げるだけでは不十分だという点である。攻撃が検出を通過し、LLM が完全に乗っ取られたと仮定しても、エージェントが不要な private resource を読めず、読めたデータも public/cloud sink へ流せない構造が必要である。

最重要結論

最も堅牢な構成は、NBAccess + SourceVault を「データ境界と情報フロー制御の正本」にし、ClawGuard 型の Task Capability 制御、ClawRouter / EdgeClaw 型の実行時 Hook 再検査、vLLM Semantic Router 型のクラウド出口検査、PrivacyChecker 型の Contextual Integrity 判定、必要に応じて OCELOT 型の累積漏洩予算を重ねる多層防御である。Prompt Injection Detector は防御の一層にすぎず、最終的な許可判断は LLM の外側で決定論的に行うべきである。

1. 調査範囲と評価基準

本レポートは、公式ドキュメント、査読論文、公式 GitHub、一次的な研究公開を優先し、候補ごとに目的、アーキテクチャ、主要コンポーネント、プライバシー境界の実装、ラベル伝播、シンク再検査、ファイル移動耐性、Prompt Injection 耐性、性能影響、運用負荷を比較する。NBAccess / SourceVault はユーザー設計・実装を含むため、公開製品と同一の成熟度として扱わず、「設計上どこまで強くできるか」と「現状実装で何が担保されるか」を区別する。

従来の評価軸は、ラベルがデータに結び付いているか、シンク直前で再検査できるか、プロベナンスと content hash で移動・複製後も追跡できるか、tool call や内部チャンネルも監査対象か、クラウド API コスト最適化と安全性を両立できるか、の五点であった。本統合版では以下の七点を追加する。

1. 外部入力を「データ」と「命令」に分離し、Untrusted content に Instruction Authority を与えないか。
2. Prompt Injection / Jailbreak 検出を入力だけでなく tool result、RAG、MCP、memory にも適用できるか。

3. ユーザーの目的から Task Capability を導出し、不要な tool/resource アクセスを決定論的に拒否できるか。
4. Private data の taint/provenance が RAG chunk、要約、LLM 出力、tool argument へ伝播するか。
5. 出力先を Sink として分類し、data label と sink clearance の関係で外部送信を拒否できるか。
6. 送信直前に local DLP / semantic guard を実行し、上流の分類ミスを再検査できるか。
7. 複数回・複数 sink への断片的な出力が累積的漏洩になることを管理できるか。

2. フレームワーク比較

NBAccess / SourceVault は「何が機密で、どこから来たデータか」を扱う層である。NBAccess は Notebook セル、ファイル、値、操作を安全カーネルの外側へ直接出さず、PrivacyLevel や ObjectSpec 等を通じて判定する設計を採る。SourceVault は source / snapshot / claim / bundle を持ち、content-addressed な参照、immutable snapshot、append-only な履歴を使ってプロベナンスを保持する。これにより、Path だけに意味を持たせるアクセス制御から、データと由来に基づくアクセス制御へ移行できる。

ClawXRouter / EdgeClaw は、実行時の経路を横断的に監視するルータ型である。三段階の privacy routing、local detector、tool/result/session hooks、dual-track memory、tool governance、sandbox を組み合わせ、セキュリティルータをコストルータより先に実行する。これはラベルを完全に保持する方式とは異なるが、送信・保存・tool 利用の境界で繰り返し再評価するため、運用中の事故を止めやすい。

vLLM Semantic Router はゲートウェイ寄りで、request から PII、jailbreak、intent、authorization 等の signal を抽出し、decision rule と plugin chain で routing や security policy を適用する。prompt がゲートウェイを必ず通る構成では有効だが、agent 内部の tool result や memory まで自動的に end-to-end で追跡するわけではないため、単独では Information Flow Control の代替にならない。

PRISM / Privacy-R1 / PrivacyRestore、AgentStealth / SEAL 等は、送信前にどの情報を local に残し、どこまで匿名化・分割・摂動するかを最適化する研究として有用である。一方、tool 権限の悪用や Private Source から Public Sink へのフローを決定論的に遮断することは主目的ではない。SecureLLM は silo 単位の権限分離に強く、EvidenceBot は local-only 運用によって外部 sink を減らせるが、いずれも別途 agent capability と sink policy が必要になる。

2.1 総合比較表

フレームワーク	位置づけ	データ境界／ラベル	実行時再検査	Prompt Injection 対応	GitLost 型耐性	主な限界
NBAccess + SourceVault	安全カーネル＋レジストリ	強：ObjectSpec / PrivacyLabel / provenance / snapshot	強く設計可能	Capability + IFC + Sink 制御を実装すれば非常に強い	★★★★★（設計完成時）	全外部通信・tool・派生物に強制点を通す必要。管理外コピーは別対策
ClawXRouter / EdgeClaw	Privacy Router＋agent hooks	中～強：内容分類＋dual-track	強：複数 Hook	入力・tool call・tool result・送信を再検査	★★★★☆	秘密ソースコード等を内容だけで S1 判定する誤分類余地
vLLM Semantic Router	Gateway / signal router	弱～中：request signal 中心	Gateway では強い	PII / jailbreak / auth signal	★★★☆☆	agent 内部 trajectory の IFC は別実装
SecureLLM	silo access 分離	中～強：silo 権限	部分的	権限外データ取得を抑制	★★★★☆	正規権限で読んだ機密データの public sink 流出は別問題
PrivacyChecker	Contextual Integrity guard	文脈・recipient 基準	候補出力で適用可能	漏洩文脈の検知に有効	★★★☆☆	決定論的な object-level IFC とは異なる
PRISM / Privacy-R1	cloud-edge privacy routing	entity/chunk 単位	部分的	IPI 自体の防御は主目的外	★★☆☆☆	tool abuse / sink flow を直接強制しない
PrivacyRestore	client-server privacy	span 中心	部分的	IPI 自体の防御は主目的外	★★☆☆☆	privacy span 外の秘密情報に弱い
AgentStealth / SEAL	local anonymization	匿名化テキスト中心	限定的	IPI 自体の防御は主目的外	★★☆☆☆	tool 権限と外部 sink 制御は別レイヤ
EvidenceBot	local RAG	retrieval context 中心	限定的	外部 write tool がなければ攻撃面小	★★★★☆※	agent 化・外部 tool 接続時は別ガードが必要
ConfAlde / PrivacyLens	評価・benchmark	該当なし	評価用途	防御ではなく検証	★☆☆☆☆	runtime enforcement ではない
IBM AI Privacy Toolkit	ML privacy toolkit	training/data privacy 中心	なし	agent IPI 防御ではない	★☆☆☆☆	モデル／データの privacy 評価が主目的

3. ラベル喪失とファイル移動に対する耐性

この論点で最も重要なのは、権限が「ファイルパス」に付いているのか、「データオブジェクトと由来」に付いているのかである。パスベースの分類では、private folder から public folder へコピーしただけで実質的なセキュリティ意味論が変化し、分類漏れがそのまま cloud 送信へつながり得る。

NBAccess / SourceVault は、少なくとも管理対象データについて、ObjectSpec、LogicalSourceId、SnapshotId、ContentHash、PrivacyLabel、Provenance、DerivedFrom 等を持たせることで、場所依存からオブジェクト依存へ寄せることができる。重要なのは、コピーでラベルが失われた場合に Public へ降格させるのではなく Unknown として扱い、Unknown から Cloud/Public への送信を fail-closed で拒否することである。

```
PrivacyLabel = Missing -> Unknown
Unknown -> Cloud/Public = DENY

Derived.PrivacyLabel = Join(InputLabels)
Derived.Provenance = Union(InputProvenance)
```

ClawXRouter / EdgeClaw は、データに恒久ラベルを結び付けるよりも、tool result や送信直前に再分類することでラベル脱落を吸収する。これは content hash 追跡の代替ではないが、実際の漏洩経路に強制点を置くため、実運用上は強い。vLLM Semantic Router は prompt 化後の出口検査として有効だが、もとのファイル由来や派生関係を持たないため、provenance-based IFC と組み合わせる方がよい。

推奨

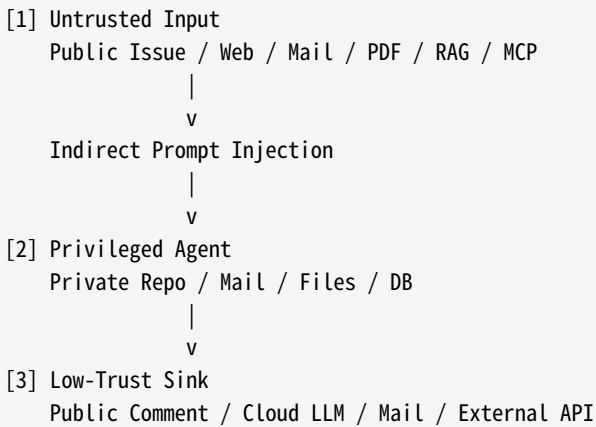
「ラベルを保持する層」と「送信直前に再判定する層」を分離する。SourceVault/NBAccess が data-bound label と provenance を保持し、agent hook が tool/message 経路を監視し、gateway が最後に PII/DLP/Prompt Guard を再検査する三重構成が現実的である。

4. プロンプトインジェクションとエージェント経由の情報漏洩

4.1 脅威モデル：GitLost 型の間接プロンプトインジェクション

間接プロンプトインジェクションでは、攻撃者はユーザーが直接入力する prompt を改変する必要がない。Web ページ、GitHub Issue、Pull Request、電子メール、PDF、RAG 文書、MCP サーバーの返答、tool result 等に命令を埋め込み、エージェントがそれを「観測データ」ではなく「実行すべき命令」と誤解するよう誘導する。

GitLost はこの問題を明確に示した事例である。公開 Issue を入力として読む agentic workflow が、組織内の private repository を読む権限と public comment を書き込む権限を同時に持つと、攻撃者の Issue 本文が agent を誘導し、private repository 由来の内容を public comment へ出力させ得る。ここで問題なのは、private repository の read 自体も public comment の write 自体も個別には正規の権限であり、危険性は「Private Source → Public Sink」という組合せで初めて現れることである。



したがって GitLost 型攻撃は、(1)未信頼入力、(2)機密資源へアクセス可能なエージェント、(3)外部または低信頼 sink、の三条件が結合したときに成立する。防御の本質は Prompt Injection を 100%検出することではなく、この三条件の少なくとも一つ、できれば複数を構造的に破壊することである。

4.2 必要な防御層

8. Trust / Provenance tagging : 入力の由来と信頼度を明示し、Public であることと Trusted であることを混同しない。
9. Instruction Authority separation : 外部データは命令権限を持たない。Public GitHub Issue は Privacy=Public でも Trust=Untrusted、InstructionAuthority=None とする。
10. Prompt Injection Guard : heuristic、classifier、local LLM 等で攻撃候補を検出する。ただし最終権限判断には使わない。
11. Task Capability Guard : ユーザー目的に必要な tool/resource だけを許可し、不要な private access を拒否する。
12. Provenance-aware IFC : 取得データと派生物に PrivacyLabel / Provenance / Taint を伝播させる。
13. Sink Clearance : Public、Cloud、Email、External API 等の sink ごとに clearance を定義し、高ラベルデータの流出を拒否する。
14. Final Egress Guard : 外部送信直前に local DLP / semantic guard を通し、誤分類を再検査する。
15. Cumulative Leakage Control : 複数回・複数 recipient への小さな開示の累積を監視する。
16. Persistent Control Defense : untrusted source 由来の control-like text を trusted memory や control file へ昇格させない。

4.3 NBAccess + SourceVault での防御

NBAccess + SourceVault の強みは、Prompt Injection そのものを完全に見抜けなくても、実行・読取・出力の情報フローを LLM の外側で決定論的に制御できる点にある。エージェントが完全に攻撃者の命令へ従ったと仮定しても、最終的なセキュリティ判断を NBAccess が担えばよい。

```

PublicIssue = {
  PrivacyLabel: Public,
  TrustLabel: Untrusted,
  InstructionAuthority: None
}
  
```

1. Issue を「データ」として読む
2. Injection が Read(privateRepo) を提案

3. Capability Gate: task に不要なら DENY
4. 読取が正規に許可されても結果は Privacy=Private
5. AddPublicComment(data) を提案
6. Sink Clearance: Private $\leq$ Public ? NO
7. DENY / NeedsApproval

これにより、Prompt Guard が失敗しても Private data の public exfiltration は成立しない。現在の連続値 PrivacyLevel を使う場合でも、派生データは入力 of 最大 PrivacyLevel を継承し、将来的に部分順序ラベルへ拡張する場合は Join 演算で上界を取る設計が自然である。

```
CanRelease(data, sink) :=
  ExplicitLabelKnown(data)
  AND data.PrivacyLabel  $\leq$  sink.Clearance
  AND TaskAllows(sink)
  AND ProvenancePolicyAllows(data, sink)
  AND LeakageBudgetAllows(data, sink)
  AND FinalLocalGuardPasses(data)
```

設計原則

LLM を信頼しない。Prompt Injection Detector も信頼しない。LLM が完全に侵害されている前提で、「何を読めるか」「何を実行できるか」「読んだデータをどこへ流せるか」を NAccess が決定論的に管理する。

4.4 ClawXRouter / EdgeClaw

EdgeClaw / ClawXRouter は、実行ライフサイクル上の複数地点に Hook を持ち、before_tool_call で file access を検査し、after_tool_call で tool result を再検査し、tool_result_persist や before_message_write で永続化内容を分離し、message_sending で出力を検査・脱機微化・取消できる。したがって GitLost 型では、private resource への tool call 前、private data 取得後、public message 送信前の複数地点で停止できる。

一方で、分類が内容中心の場合、未公開ソースコードや研究上の秘密のように「文字列だけでは PII や secret pattern に見えない機密」を S1 と誤判定する余地がある。したがって EdgeClaw 型の runtime content guard は、NAccess/SourceVault 型の provenance label と組み合わせることで強くなる。

4.5 vLLM Semantic Router

vLLM Semantic Router は PII detection、Prompt Guard、role-based authorization 等の signal を routing decision へ組み込めるため、入口および gateway 層の防御として有用である。GitLost 型の悪性 Issue が gateway を通るなら jailbreak/prompt-injection signal として検知できる可能性がある。

ただし Semantic Router は agent の tool execution framework そのものではない。Web/RAG/MCP/tool result が router を迂回して context へ入り、その後 agent が private tool を呼び出す構成では、request-level guard だけでは不十分である。NAccess と組み合わせる場合は「Semantic Router = security signal provider」「NAccess = hard policy authority」と分担するのが適切である。

4.6 研究系 privacy routing / 匿名化方式

PRISM、Privacy-R1、PrivacyRestore、AgentStealth、SEAL は、主として cloud へ送信する情報量や匿名化の仕方を最適化する。したがって privacy-utility tradeoff には有効だが、Prompt Injection による tool misuse を直接止める機構ではない。とくに Privacy-R1 のような soft routing は、task utility のために一部の機密情報を remote へ送る選択を許容し得るため、hard boundary としては NBAccess 等の下位に置くべきである。

4.7 SecureLLM、PrivacyChecker、EvidenceBot、IBM AI Privacy Toolkit

SecureLLM は silo 単位の access separation により、そもそも権限のない private data を agent が取得できない構造を作れる点で強い。しかし agent 自身に正規の private access が付与されている場合、そのデータを public sink へ出すことまで自動的に防ぐとは限らない。

PrivacyChecker は Contextual Integrity に基づき、sender、recipient、context、data type の関係から開示の妥当性を評価するため、「private repository 由来情報を public issue へ送る」といった文脈的不整合を捉える補助層として有用である。一方、object-level な決定論的 IFC とは性質が異なるため、hard policy と併用する。

EvidenceBot のような local-only RAG は、external write tool を持たない構成では GitLost に必要な sink を削減できる。ただし agent 化して GitHub、Email、Web API 等を接続した瞬間に別の防御が必要になる。IBM AI Privacy Toolkit は ML model や training data の privacy risk assessment、anonymization、data minimization 等が主目的であり、agent runtime の Prompt Injection defense や tool boundary enforcement の代替ではない。

4.8 専用防御技術から得られる設計要素

ClawGuard は、ユーザー目的から task-specific access constraints を導出し、各 tool-call boundary で決定論的に検査する。これは NBAccess へそのまま取り込める考え方であり、Prompt Injection を「malicious text の検出問題」から「不正な action の実行可否問題」へ変換する。

OCELOT は、単発の release ではなく trajectory 全体の累積的な inference leakage を予算化する。個々の出力が無害でも、複数 sink が結託すれば秘密を推定できるケースを扱うため、将来の NBAccess では sink trust に応じた leakage budget、deterministic verifier、tamper-evident ledger を検討する価値がある。

ClawTrojan / DASGuard は、untrusted file や tool output 中の命令が memory や control file へ書き込まれ、後の session で再実行される「永続的制御」を問題化している。これに対して SourceVault では、TrustLabel と InstructionAuthority を PrivacyLabel とは別軸にし、外部 source 由来の control-like text を trusted instruction へ昇格させないことが重要である。

4.9 プロンプトインジェクション耐性比較表

システム	Injection 検出	Instruction/Data 分離	Tool Capability	Tool 結果再検査	IFC / Taint	Sink 制御	評価
NBAccess + SourceVault	外付け推奨	強く実装可能	強く実装可能	強く実装可能	最も強く設計可能	強く実装可能	★★★★★※
EdgeClaw / ClawXRouter	rule + local model	Guard Prompt 等	tool governor / file guard	あり	内容分類中心	message_sending 等	★★★★☆
vLLM Semantic Router	PII / jailbreak	request signal 中心	tool selection あり	agent 内部は外部実装	弱	gateway routing	★★★☆☆
SecureLLM	直接対象外	silos 境界	権限ベース	なし	silos 単位	recipient flow は別	★★★☆☆
PrivacyChecker	privacy violation 検知	文脈的に評価	間接的	候補出力で可	Contextual Integrity	recipient-aware	★★★☆☆
PRISM / Privacy-R1	なし	なし	なし	なし	soft routing	cloud/edge	★~★★
PrivacyRestore	なし	なし	なし	なし	span 除去	cloud 前処理	★★☆☆☆
AgentStealth / SEAL	なし	なし	なし	なし	匿名化	cloud 前処理	★★☆☆☆
EvidenceBot	なし	local QA 限定	外部 tool なしなら小	RAG 内	弱	local-only なら小	★★★★☆※
ConfAlde / PrivacyLens	評価のみ	評価	なし	trajectory 評価	なし	なし	★☆☆☆☆
IBM AI Privacy Toolkit	なし	なし	なし	なし	ML privacy	なし	★☆☆☆☆

5. 性能影響と運用コスト

コスト最適化を前面に出しているのは ClawXRouter / EdgeClaw / vLLM Semantic Router である。ClawXRouter は privacy 判定を高優先度で実行し、安全性が確認された S1 request にのみ cost-aware routing を適用するため、「安全性とコスト最適化の順序」を明示している点が重要である。

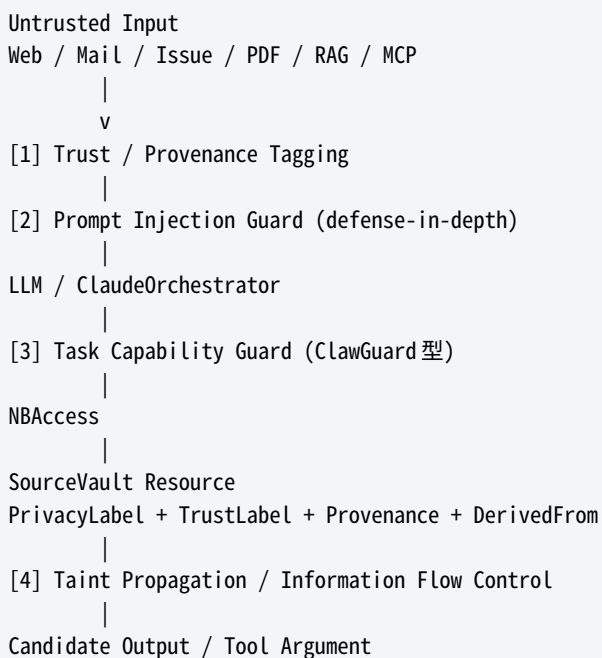
Prompt Injection 対策を追加すると、入力分類、tool-call 検査、tool-result 検査、egress 検査の各段階でレイテンシが増える。したがって、すべてを大規模 LLM で判定するのではなく、(1)deterministic rule / regex、(2)軽量 local classifier、(3)必要時のみ local LLM、(4)hard policy decision の順に階層化するのが望ましい。hard policy は高速なラベル比較と capability 判定にし、LLM 判定は「分類を厳しくする」方向にのみ使うべきである。

NBAccess / SourceVault は、privacy label、provenance、sink clearance の比較自体は軽量にできるが、SourceVault ingest、OCR、semantic reinspection、local guard model 等は別コストになる。重要なのは、セキュリティルータを cost router より先に置き、private data を「安いから cloud へ送る」といった逆転を起こさないことである。

OCELOT 型の累積漏洩予算は、単発判定より状態管理が増えるため、すべてのワークロードへ一律適用するより、high-risk sink や高機密 task へ限定導入する方が現実的である。また Prompt Injection 検出は false positive が運用阻害につながるため、Deny / NeedsApproval / LocalOnly / RedactAndCloud 等の複数アクションを用意するとよい。

6. 推奨アーキテクチャ

最も現実的な推奨は、NBAccess + SourceVault をデータ境界と情報フロー制御の正本にし、ClawGuard 型の Task Capability Guard、ClawXRouter / EdgeClaw 型の runtime hook、vLLM Semantic Router 型の gateway guard を重ねる構成である。さらに Contextual Integrity 層と累積漏洩制御を必要に応じて追加する。



```

|
[5] Contextual Integrity Check
|
[6] Sink Clearance Check
|
[7] Final Local DLP / Semantic Guard
|
[8] Cumulative Leakage Budget (high-risk)
|
Permit / Deny / NeedsApproval / LocalOnly
|
Cloud LLM / Email / GitHub / External API

```

6.1 推奨メタデータ

```

LogicalSourceId
SnapshotId
ContentHash
PrivacyLabel
TrustLabel
InstructionAuthority
Provenance
DerivedFrom[]
TaintSet[]
SourceType
Owner
AllowedSinks[]

```

PrivacyLabel と TrustLabel は必ず分離する。Public 情報は機密ではないが、攻撃者が作成できる Public Issue は信頼できる命令ではない。したがって、Public を Trusted の同義語として扱ってはならない。

```

PublicIssue.PrivacyLabel      = Public
PublicIssue.TrustLabel       = Untrusted
PublicIssue.InstructionAuthority = None

```

6.2 決定規則

```

CapabilityAllowed(action, resource, task) :=
  ToolInTaskCapability
  AND ResourceReadAllowed
  AND ActionAllowed

CanWrite(data, sink) :=
  LabelKnown(data)
  AND data.PrivacyLabel <= sink.Clearance
  AND task.Allows(sink)
  AND provenance.Allows(data, sink)

Unknown -> Cloud/Public = DENY

```

local DLP や semantic guard は、機密度を自動的に上げることは許しても、下げることは原則として許さない。Declassification は明示的なポリシー、監査ログ、必要に応じて human approval を伴うべきである。

6.3 Memory と永続化

Prompt Injection は一回の応答で終わらず、memory、workspace、skill file、control file へ悪性命令が保存され、後の session で発火する可能性がある。したがって、外部 source 由来の text を memory へ保存する場合も TrustLabel と Provenance を保持し、InstructionAuthority を自動付与しない。control-like text を trusted policy へ昇格させる場合は、owner/system 由来であることを検証する。

7. 導入計画

短期：実行時フックと Sink 直前ガード

- before_prompt_build / before_tool_call / after_tool_call / before_message_write / message_sending 相当の強制点を ClaudeOrchestrator 経路へ追加する。
- Public/Web/Mail/PDF/RAG/MCP を既定で Untrusted とし、InstructionAuthority=None にする。
- 外部送信を一つの gateway 経路へ集約し、PII/DLP/Prompt Guard を再検査する。
- Unknown ラベルの cloud/public 送信を fail-closed にする。

中期：Data-bound label と Capability Security

- LogicalSourceId / SnapshotId / ContentHash / Provenance / DerivedFrom を SourceVault 正本へ統合する。
- RAG chunk、要約、LLM 生成物、tool argument へ PrivacyLabel/Taint を伝播させる。
- ユーザー目的から Task Capability を導出し、タスク外の private tool/resource アクセスを拒否する。
- Public/Cloud/Email/GitHub 等を Sink として clearance 定義する。

長期：累積漏洩・OS 隔離・機密推論

- high-risk task に OCELOT 型の sink-trust-weighted leakage budget を導入する。
- workspace/memory の persistent control を DASGuard 型の origin 検証で防ぐ。
- ClawXSandbox や OS MAC/ACL と連携し、NBAccess の論理ポリシーを OS 強制へ接続する。
- 最高機密ワークロードでは confidential inference / TEE 等を検討する。

8. 結論

クラウド LLM とローカル LLM をプライバシー境界で切り分ける問題は、単なるモデルルーティングではない。データの機密度を判定して cloud/local を選ぶだけでは、ファイル移動によるラベル脱落、tool result 経由の漏洩、Prompt Injection による正規権限の悪用、複数 sink への累積的開示を防げない。

この観点では、NBAccess + SourceVault は「データと由来を保持する層」、ClawXRouter / EdgeClaw は「実行中の経路を再検査する層」、vLLM Semantic Router は「外部 LLM 出口を監視する層」として相補的である。さらに ClawGuard 型の Task Capability Control を加えることで、不正な resource access を実行前に止められる。

GitLost 型攻撃への最も重要な回答は、Prompt Injection を見抜くことではなく、LLM が完全に騙されても Private Source から Public Sink へのフローが成立しないことを保証することである。したがって NBAccess の最終的な位置づけは、単なる Privacy Router ではなく、Agent Information-Flow Security Kernel へ拡張するのが妥当である。

最終推奨

data-bound label + task capability control + provenance-aware IFC + sink clearance + runtime hook
 reinspection + gateway egress guard を基本構成とする。Prompt Injection 検出は補助的防御とし、最終の
 Permit / Deny / NeedsApproval は LLM の外側で決定論的に行う。

参考資料

- [1] Noma Security, “GitLost: How We Tricked GitHub’s AI Agent into Leaking Private Repos,” 2026-07-06.
<https://noma.security/blog/gitlost-how-we-tricked-githubs-ai-agent-into-leaking-private-repos/>
- [2] GitHub Docs, “About GitHub Agentic Workflows.”
<https://docs.github.com/en/enterprise-cloud@latest/copilot/concepts/agents/about-github-agentic-workflows>
- [3] OpenBMB, “EdgeClaw / ClawXRouter,” GitHub. <https://github.com/OpenBMB/EdgeClaw>
- [4] vLLM Project, “Semantic Router,” GitHub. <https://github.com/vllm-project/semantic-router>
- [5] Liu et al., “vLLM Semantic Router: Signal Driven Decision Routing for Mixture-of-Modality Models,” arXiv:2603.04444, 2026.
- [6] Zhao et al., “ClawGuard: A Runtime Security Framework for Tool-Augmented LLM Agents Against Indirect Prompt Injection,” arXiv:2604.11790, 2026.
- [7] Xie and Li, “OCELOT: Inference-Leakage Budgets for Privacy-Preserving LLM Agents,” arXiv:2606.12341, 2026.
- [8] Tan et al., “From Prompt Injection to Persistent Control: Defending Agentic Harness Against Trojan Backdoors,” arXiv:2605.31042, 2026.
- [9] Wang et al., “Privacy in Action: Towards Realistic Privacy Mitigation and Evaluation for LLM-Powered Agents,” Findings of EMNLP 2025.
- [10] IBM Research, “AI Privacy Toolkit.” <https://research.ibm.com/publications/ai-privacy-toolkit>
- [11] 元レポート：『クラウド LLM とローカル LLM をプライバシー境界で切り分ける LLM 呼び出しフレームワーク詳細比較レポート』。